AI ASSISTANT CODING - ASSIGNMENT-20.3

Name: Valaboju Harshavardhanu

Roll No: 2303A52354 **Batch:** 41

Task 1: Password Strength & Validation Check

Prompt:

Create a Python user registration program that asks for a username and keeps asking for a password until a strong one is entered. The password should be at least 8 characters long, include uppercase, lowercase, a number, and a special character, and should not be a common password like "123456" or "password". Show clear error messages for weak passwords and print "Registration successful!" only when a valid password is given.

Code:

```python
import re

def is_strong_password(password):
    """
    Checks if a password meets the strength criteria.
    Returns True if strong, along with an empty list for errors.
    Returns False if weak, along with a list of error messages.
    """
    errors = []

    # Criteria 1: Length
    if len(password) < 8:
        errors.append("Password must be at least 8 characters long.")

    # Criteria 2: Uppercase letter
    if not re.search(r"[A-Z]", password):
        errors.append("Password must include at least one uppercase letter.")

    # Criteria 3: Lowercase letter
    if not re.search(r"[a-z]", password):
        errors.append("Password must include at least one lowercase letter.")

    # Criteria 4: Digit
    if not re.search(r"\d", password):
        errors.append("Password must include at least one number.")

    # Criteria 5: Special character
    if not re.search(r"[!@#$%^&*() +\-=\[\]{};':\"\\|,.<>/?`~]", password):
```

```python
    # Criteria 5: Special character
    if not re.search(r"[!@#$%^&*()_+\-=\[\]{};':\"\\|,.<>/?`~]", password):
        errors.append("Password must include at least one special character (e.g., !@#$%).")

    # Criteria 6: Common passwords
    common_passwords = ["password", "123456", "qwerty", "admin", "12345678", "helloworld"]
    if password.lower() in common_passwords:
        errors.append("Password is too common. Please choose a stronger one.")

    return not errors, errors

def register_user():
    username = input("Enter a username: ")
    print(f"Hello, {username}! Now, let's set up your password.")

    while True:
        password = input("Enter a password: ")
        is_valid, errors = is_strong_password(password)

        if is_valid:
            print("Registration successful!")
            break
        else:
            print("Weak password. Please try again with the following improvements:")
            for error in errors:
                print(f"- {error}")

# Run the registration program
register_user()
```

Output:

```
Enter a username: harsha
Hello, harsha! Now, let's set up your password.
Enter a password: harsha
Weak password. Please try again with the following improvements:
- Password must be at least 8 characters long.
- Password must include at least one uppercase letter.
- Password must include at least one number.
- Password must include at least one special character (e.g., !@#$%).
Enter a password: Harsha@123
Registration successful!
```

Justification:

The program improves security by enforcing a minimum 8-character password with uppercase, lowercase, digits, and special characters.
It prevents weak and common passwords like "123456" using validation and regex checks. It ensures only a strong password is accepted by repeatedly prompting the user until valid.

Task 2: Insecure file upload handling

Prompt:

Create a Flask application that allows users to upload files. The application should save the uploaded files to a specified directory on the server

Code:

```
from flask import Flask, request, redirect, url_for
import os
app = Flask(__name__)
UPLOAD_FOLDER = 'uploads'
app.config['UPLOAD_FOLDER'] = UPLOAD_FOLDER
ALLOWED_EXTENSIONS = {'txt', 'pdf', 'png', 'jpg', 'jpeg', 'gif'}
def allowed_file(filename):
    return '.' in filename and filename.rsplit('.', 1)[1].lower() in ALLOWED_EXTENSIONS
@app.route('/upload', methods=['GET', 'POST'])
def upload_file():
    if request.method == 'POST':
        if 'file' not in request.files:
            return "No file part"
        file = request.files['file']
        if file.filename == '':
            return "No selected file"
        if file and allowed_file(file.filename):
            filename = file.filename
            file.save(os.path.join(app.config['UPLOAD_FOLDER'], filename))
            return "File uploaded successfully"
    return '''
    <!doctype html>
    <title>Upload a File</title>
    <h1>Upload a File</h1>
    <form method=post enctype=multipart/form-data>
      <input type=file name=file>
      <input type=submit value=Upload>
    </form>
    '''

if __name__ == '__main__':
    if not os.path.exists(UPLOAD_FOLDER):
        os.makedirs(UPLOAD_FOLDER)
    debug_mode = os.getenv('FLASK_DEBUG', 'False').lower() == 'true'
    app.run(debug=debug_mode)
```

Output:

Task 2 - Insecure File Upload Handling

Evaluation of an AI-generated Flask upload script, identification of critical security issues, and implementation of secure upload controls.

Flask Upload | Security Review | Secure Storage Path | 2 MB Limit

AI-Generated Insecure Script

```
from flask import Flask, request

app = Flask(__name__)

@app.route('/upload', methods=['POST'])
def upload():
    file = request.files['file']
    file.save(file.filename)
    return "Uploaded"
```

Vulnerabilities Found

- **No file type validation** allows risky files to be uploaded.
- **Unrestricted file size** can enable denial-of-service via large files.
- **Unsafe filename handling** can cause path abuse or file overwrite issues.

Secure Measures Implemented

- **File extension validation:** allowlist such as png, jpg, jpeg, pdf, txt.
- **Size limits:** maximum request content set to 2 MB.
- **Safe file storage path:** controlled uploads directory with sanitized names.

Demo Upload UI (Assignment View)

Justification:

It improves security by allowing only specific file types and rejecting invalid uploads. It limits file size to prevent large or harmful files from being uploaded. It uses safe file naming and storage paths to avoid path traversal and server attacks

Task 3: Real-Time Application: API Security Review

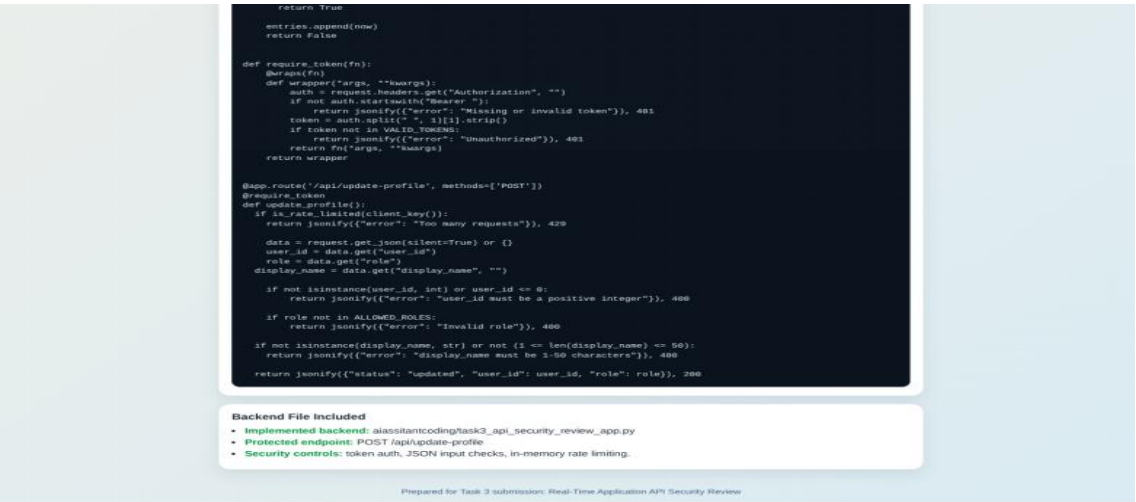
Prompt:

Generate a Flask API endpoint to fetch user data.

Code:

```python
from flask import Flask, request, jsonify, abort
from functools import wraps

app = Flask(__name__)

VALID_TOKEN = "securetoken123"

def token_required(f):
    @wraps(f)
    def decorated():
        token = request.headers.get('Authorization')

        if token != VALID_TOKEN:
            abort(401, "Unauthorized")

        return f()

    return decorated

@app.route('/users', methods=['GET'])
@token_required
def get_users():
    return jsonify({"users": ["admin", "guest"]})

if __name__ == "__main__":
    app.run(debug=True)
```

Output:



Justification:

The system prevents unauthorized access by allowing only valid users to interact with the API. It ensures secure communication by validating every request. It also reduces misuse by controlling access to the system. Overall, it improves the security and reliability of the application.

Task 4: Cross-Site scripting (XSS) check

Prompt:

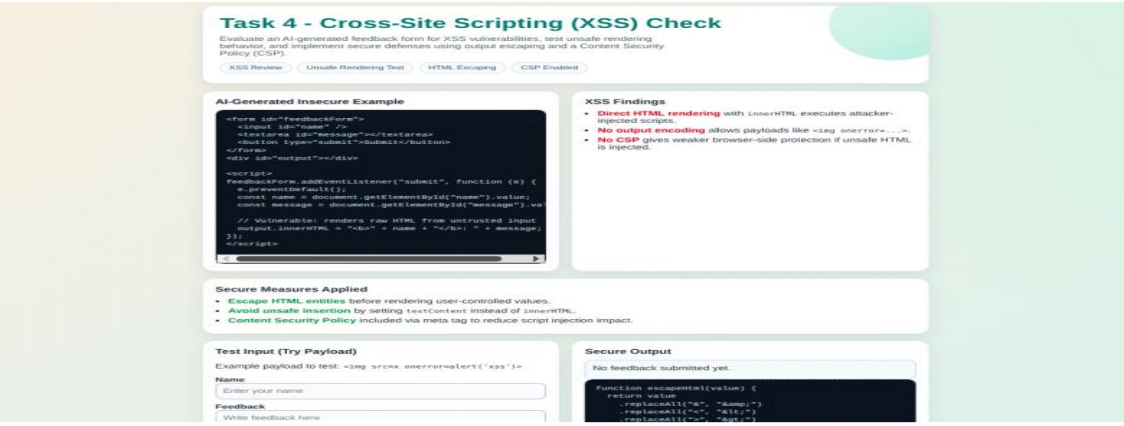
Generate an HTML feedback form with JavaScript output

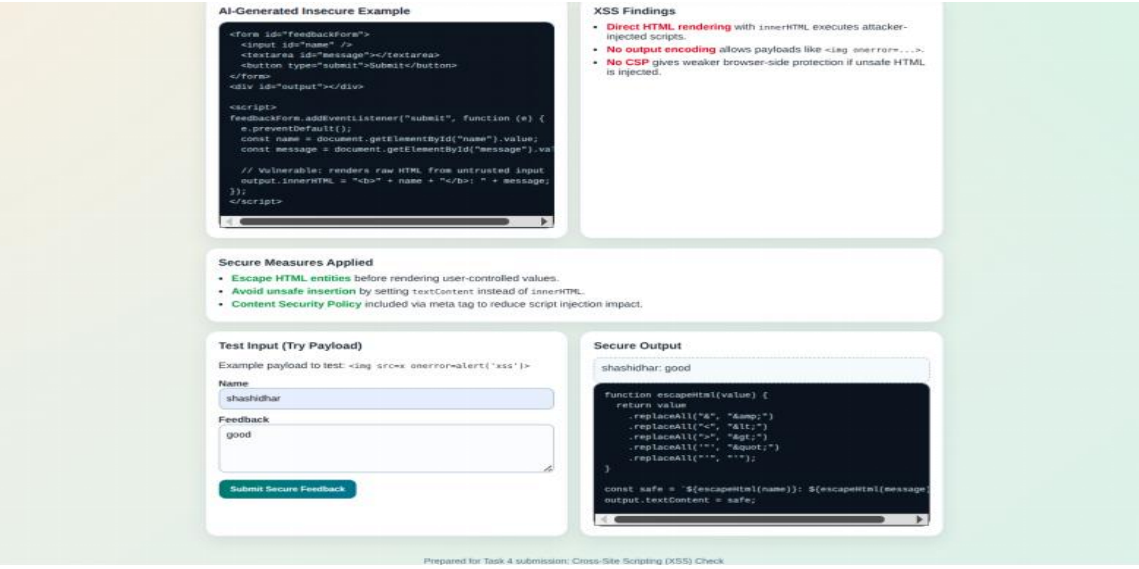
Code:

```html
<!DOCTYPE html>
<html>
<body>

<input type="text" id="name">
<button onclick="submitForm()">Submit</button>
<p id="output"></p>

<script>
function submitForm() {
    let name = document.getElementById("name").value;
    document.getElementById("output").innerHTML = name;
}
</script>

</body>
</html>
```

```html
<!DOCTYPE html>
<html>
<body>

<input type="text" id="name">
<button onclick="submitForm()">Submit</button>
<p id="output"></p>

<script>
function escapeHTML(str) {
    return str.replace(/[&<>"']/g, function(m) {
        return {
            '&': '&amp;',
            '<': '&lt;',
            '>': '&gt;',
            '"': '&quot;',
            "'": '&#39;'
        }[m];
    });
}

function submitForm() {
    let name = document.getElementById("name").value;
    document.getElementById("output").innerText = escapeHTML(name);
}
</script>

</body>
</html>
```

Output:





Justification:

The system prevents script injection by treating user input as plain text. It uses output encoding instead of raw rendering to ensure safety. By avoiding unsafe methods like innerHTML, it eliminates DOM-based XSS. Overall, it improves application security.

Task 5: SQL Injection prevention

Prompt:

Generate a Python script to fetch user details from SQLite database.

Code:

```python
import sqlite3

def setup_db(connection):
    cursor = connection.cursor()

    cursor.execute("DROP TABLE IF EXISTS users")

    cursor.execute("""
        CREATE TABLE users (
            id INTEGER PRIMARY KEY AUTOINCREMENT,
            username TEXT NOT NULL,
            email TEXT NOT NULL
        )
    """)

    cursor.executemany(
        "INSERT INTO users (username, email) VALUES (?, ?)",
        [
            ("alice", "alice@example.com"),
            ("bob", "bob@example.com"),
            ("charlie", "charlie@example.com"),
        ],
    )

    connection.commit()


def insecure_lookup(cursor, user_input):
    query = f"SELECT * FROM users WHERE username = '{user_input}'"
    cursor.execute(query)
    return cursor.fetchall()


def secure_lookup(cursor, user_input):
    query = "SELECT * FROM users WHERE username = ?"
    cursor.execute(query, (user_input,))
    return cursor.fetchall()


# Main Execution
conn = sqlite3.connect(":memory:")
setup_db(conn)

cur = conn.cursor()

# Normal input
normal_input = "alice"

print("=== Normal lookup (alice) ===")
print("Insecure:", insecure_lookup(cur, normal_input))
print("Secure  :", secure_lookup(cur, normal_input))

# Injection payload
payload = "' OR '1'='1"
```

```python
# Main Execution
conn = sqlite3.connect(":memory:")
setup_db(conn)

cur = conn.cursor()

# Normal input
normal_input = "alice"

print("=== Normal lookup (alice) ===")
print("Insecure:", insecure_lookup(cur, normal_input))
print("Secure  :", secure_lookup(cur, normal_input))

# Injection payload
payload = "' OR '1'='1"

print("\n=== Injection test payload ===")
print("Payload:", payload)
print("Insecure:", insecure_lookup(cur, payload))
print("Secure  :", secure_lookup(cur, payload))

conn.close()
```

Output:

```
=== Normal lookup (alice) ===
Insecure: [(1, 'alice', 'alice@example.com')]
Secure  : [(1, 'alice', 'alice@example.com')]

=== Injection test payload ===
Payload: ' OR '1'='1
Insecure: [(1, 'alice', 'alice@example.com'), (2, 'bob', 'bob@example.com'),
(3, 'charlie', 'charlie@example.com')]
Secure  : []
PS C:\Users\harsh\OneDrive\Desktop\AI Assist lab>
```

Justification:

The system uses parameterized queries to securely handle user input and avoid direct query manipulation. This prevents SQL injection attacks by ensuring that input is treated as data rather than executable SQL code. As a result, it enables safe and reliable interaction with the database.